



Insomnia

Projektowanie, testowanie i utrzymanie REST API

Szkolenie

- Plan szkolenia i cele
- Aktualne obowiązki
- Pytania, dyskusje, potrzeby
- Elastyczny program szkoleniowy

Ja

Mateusz Kulesza

- Senior Software Developer
- Solution Architect
- Enterprise Architect
- Consultant and Trainer

Ty

uczestnik

Kilka szybkich pytań:

- Twoje codzienne obowiązki
- Technologie i doświadczenie
- Poprzednia styczność z tematem
- Twoje cele na to szkolenie
- Obszary lub tematy szczegółowe

Wprowadzenie do REST API

Podstawy HTTP, REST i JSON

Czym jest API i architektura REST

- API – kontrakt umożliwiający komunikację między systemami
- REST – styl architektoniczny oparty na zasobach i protokole HTTP
- Klient–serwer – rozdzielenie aplikacji od dostawcy danych
- Zasoby identyfikowane przez URI, operacje przez metody HTTP
- Komunikacja **bezstanowa** – każde żądanie kompletne
- Najczęstszy format wymiany danych: **JSON**

Zasady REST:

- Jednolity interfejs i przewidywalne URI
- Rzeczowniki zamiast czasowników: `/users` , nie `/getUsers`
- Liczba mnoga dla kolekcji, zagnieżdżenia dla relacji

```
GET /users/42/orders    → zamówienia użytkownika 42
POST /users              → utworzenie nowego użytkownika
```

Przykłady zastosowań API

- Aplikacje mobilne i web – pobieranie danych z backendu
- Integracje – łączenie systemów (CRM, ERP, sklep)
- Płatności – Stripe, PayU, Przelewy24
- Mapy i geolokalizacja – Google Maps, OpenStreetMap
- Mikroustugi – komunikacja między serwisami
- IoT – urządzenia raportujące dane do chmury
- AI / LLM – API modeli (OpenAI, Anthropic)

Typowy scenariusz – płatność:

1. Frontend wysyła `POST /payments` z kwotą
2. Backend woła API operatora (Stripe)
3. Odpowiedź z `paymentId` i statusem
4. Klient odpytuje `GET /payments/{id}` o wynik

```
{ "amount": 4900, "currency": "PLN", "status": "paid" }
```

REST vs GraphQL vs SOAP

- REST – prostota, cache, najpopularniejszy
- GraphQL – elastyczne zapytania, jedno żądanie
- SOAP – formalny, korporacyjny, transakcje

Kiedy co wybrać:

- REST – publiczne API, prosty CRUD, mocny cache
- GraphQL – złożone widoki, wiele klientów, mobilki
- SOAP – bankowość, systemy legacy, gwarancje transakcji

Cecha	REST	GraphQL	SOAP
Format	JSON	JSON	XML
Endpointy	wiele	jeden	jeden
Pobieranie	over/under-fetch	dokładnie pola	sztywne
Kontrakt	OpenAPI	schema	WSDL
Próg wejścia	niski	średni	wysoki

Klient–serwer, bezstanowość, cache

- Klient–serwer – klient pyta, serwer odpowiada; jasny podział ról
- Bezstanowość – serwer nie pamięta sesji; każde żądanie kompletne
- Stan (np. token) przesyłany w każdym żądaniu → łatwa skalowalność
- Cache – odpowiedzi można buforować, gdy oznaczone jako cacheable
- Korzyści: mniej obciążenia, szybsze odpowiedzi, mniejsze koszty

Sterowanie cache:

- `Cache-Control: max-age=3600` – ważne przez godzinę
- `ETag` – odcisk wersji zasobu
- `Expires` – data wygaśnięcia

```
GET /users/42      → 200 OK, ETag: "ab12"  
GET /users/42      → If-None-Match: "ab12"  
                   ← 304 Not Modified (z cache)
```

Metody HTTP

- GET /users – lista, GET /users/42 – jeden
- POST /users – dodanie z body JSON
- PUT zastępuje całość, PATCH tylko wybrane pola
- DELETE usuwa zasób

```
PATCH /users/42
Content-Type: application/json

{ "email": "nowy@example.com" }
```

Metoda	Zastosowanie	Bezpieczna	Idempotentna
GET	pobieranie zasobu	✓	✓
POST	tworzenie zasobu	✗	✗
PUT	pełna aktualizacja	✗	✓
PATCH	częściowa zmiana	✗	✗
DELETE	usunięcie zasobu	✗	✓

Idempotentność i metody bezpieczne

- Bezpieczne – nie zmieniają stanu: GET , HEAD , OPTIONS
- Idempotentne – wielokrotne wywołanie = ten sam efekt
- PUT i DELETE idempotentne, POST nie
- Dzięki temu klient może ponawiać żądania po błędzie sieci
- Trik: nagłówek Idempotency-Key dla bezpiecznych
ponowień POST

Metoda	Bezpieczna	Idempotentna
GET	✓	✓
PUT	✗	✓
DELETE	✗	✓
POST	✗	✗

Statusy odpowiedzi HTTP

- 2xx – sukces: 200 OK , 201 Created , 204 No Content
- 3xx – przekierowanie: 301 , 304 Not Modified
- 4xx – błąd klienta: 400 , 401 , 403 , 404 , 409 , 422
- 5xx – błąd serwera: 500 , 502 , 503

Kod	Znaczenie	Typowa sytuacja
200	OK	poprawne GET
201	Utworzono	po POST z Location
400	Złe żądanie	błędny JSON / parametry
401	Brak autoryzacji	brak/niepoprawny token
404	Nie znaleziono	zasób nie istnieje
500	Błąd serwera	wyjątek po stronie backend

Kody błędów – przykłady

- 400 – błędne body lub parametry
- 401 – brak/niepoprawny token (uwierzytelnienie)
- 403 – brak uprawnień (autoryzacja)
- 404 – zasób nie istnieje
- 409 – konflikt (np. duplikat)
- 422 – walidacja danych nie przeszła
- 429 – zbyt wiele żądań (rate limit)

```
{  
  "error": "validation_failed",  
  "message": "Pole 'email' jest wymagane",  
  "status": 422  
}
```

Nagłówki HTTP

- `Content-Type` – format body, np. `application/json`
- `Accept` – format oczekiwany przez klienta
- `Authorization` – uwierzytelnianie, np. `Bearer <token>`
- `Cache-Control` – reguły buforowania, np. `no-cache`
- `User-Agent` – identyfikacja klienta
- `ETag` – wersja zasobu do walidacji cache

Żądanie i odpowiedź:

```
GET /users/42 HTTP/1.1
Accept: application/json
Authorization: Bearer eyJhbGci...
```

```
HTTP/1.1 200 OK
Content-Type: application/json
ETag: "ab12"
```

Bezpieczeństwo: HTTPS i CORS

- HTTPS – szyfrowanie TLS, ochrona danych w transzycie
- Zapobiega podsłuchowi i modyfikacji ruchu
- CORS – kontrola dostępu między domenami w przeglądarce
- Serwer określa, kto może wywołać API
- Bez nagłówka brak dostępu z innej domeny

Dobre praktyki:

- Zawsze HTTPS, tokeny w `Authorization`, nie w URL
- Ograniczaj origin do zaufanych domen
- Preflight `OPTIONS` dla niestandardowych żądań

```
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Methods: GET, POST, PUT
Access-Control-Allow-Headers: Authorization, Content-Type
```



Format danych: JSON

- Lekki, czytelny format wymiany danych
- Typy: tekst, liczba, `true/false`, `null`, obiekt, tablica
- Klucze w cudzysłowach, bez przecinka na końcu
-  camelCase, spójne nazwy, daty w ISO 8601
-  waliduj wejście; nie zwracaj danych wrażliwych

```
{  
  "id": 42,  
  "name": "Anna",  
  "active": true,  
  "roles": ["admin", "user"]  
}
```


JSON vs XML

Cecha	JSON	XML
Czytelność	wysoka	średnia
Rozmiar	mniejszy	większy
Typy danych	natywne	tekst
Schemat	JSON Schema	XSD
Użycie	REST, web	SOAP, legacy

- JSON dominuje w REST API; XML w systemach korporacyjnych

```
{ "user": { "id": 1, "name": "Anna" } }
```

```
<user><id>1</id><name>Anna</name></user>
```

Projektowanie REST API

Zasoby, URI, błędy i wersjonowanie

Identyfikacja zasobów

- Zasób = rzeczownik (np. `user` , `order` , `product`), nie czynność
- Zawsze nazwy w liczbie mnogiej: `/users` , `/orders`
- Relacje przez zagnieżdżenie: `/users/42/orders`
- Jeden zasób = unikalny identyfikator: `/users/42`
- Kolekcja zwraca listę, element pojedynczy obiekt
- Czasowniki w URI to sygnał błędnego projektu
- Akcję wyraża metoda HTTP, nie ścieżka

✗ Źle

✓ Dobrze

`/getUser?id=42`

GET `/users/42`

`/createOrder`

POST `/orders`

`/deleteUser/42`

DELETE `/users/42`

`/listOrders`

GET `/orders`

`/user/42/cancel`

POST `/users/42/cancel`

Wskazówka: projektuj URI tak, by były czytelne bez dokumentacji.

RESTful URI – dobre praktyki

- Małe litery, słowa łączone myślnikiem: `/order-items`
- Rzeczowniki w liczbie mnogiej, bez czasowników
- Hierarchia relacji: `/users/42/orders/7`
- Bez rozszerzeń plików (`.json`) – format z `Accept`
- Filtry w query string: `?status=active`
- Adresy stabilne i przewidywalne
- Maks. 2 poziomy zagnieżdżenia – głębiej trudno czytać
- Bez `trailing slash` na końcu URI

Element	Przykład
Kolekcja	<code>/products</code>
Element	<code>/products/100</code>
Zagnieżdżenie	<code>/products/100/reviews</code>
Filtr	<code>/products?tag=sale</code>

Złe: `/Products/getAll.json` **Dobre:** `GET /products` +
`Accept: application/json`

Metody HTTP – semantyka

- GET – pobranie, bez efektów ubocznych (idempotentne)
- POST – utworzenie zasobu (nieidempotentne)
- PUT – pełna aktualizacja/zamiana (idempotentne)
- PATCH – częściowa aktualizacja (np. jedno pole)
- DELETE – usunięcie (idempotentne)
- *Bezpieczna* = nie zmienia stanu; *idempotentna* = powtórka bez skutku
- Statusy: 200, 201 Created, 204 No Content, 404

Metoda	Idemp.	Bezp.	Przykład
GET	✓	✓	GET /users
POST	✗	✗	POST /users
PUT	✓	✗	PUT /users/42
PATCH	✗	✗	PATCH /users/42
DELETE	✓	✗	DELETE /users/42

```
PATCH /users/42
{ "email": "nowy@example.com" }
```

Obsługa błędów

- Statusy zgodne z HTTP: 4xx klient, 5xx serwer
- Spójny format komunikatu (np. RFC 7807 / Problem Details)
- Czytelny kod błędu + opis dla człowieka i maszyny
- Nigdy nie zwracaj 200 przy błędzie
- Nie ujawniaj wrażliwych szczegółów serwera

```
{  
  "type": "validation_error",  
  "status": 400,  
  "title": "Nieprawidłowe dane",  
  "errors": [{ "field": "email", "msg": "wymagane" }]  
}
```

Kod	Znaczenie
400	Błędne żądanie
401/403	Brak autoryzacji/dostępu
404	Nie znaleziono
422	Błąd walidacji

Filtrowanie, sortowanie, paginacja, wersjonowanie

- Filtrowanie: `?category=books&status=active`
- Sortowanie: `?sort=price` / `?sort=-price` (malejąco)
- Paginacja: `?page=2&limit=20` lub kursor `?cursor=abc`
- Wersjonowanie: `/v1/users` lub nagłówek `Accept`
- Limity i wartości domyślne chronią serwer
- Metadane w odpowiedzi: `total` , `page` , `next`
- Kursor lepszy od offsetu przy dużych zbiorach

Funkcja	Przykład
Filtr	<code>?status=active</code>
Sort	<code>?sort=-created</code>
Strona	<code>?page=3&limit=50</code>
Wersja	<code>/v2/orders</code>

```
GET /products?category=books&sort=-price&page=1&limit=20
Accept: application/vnd.api.v1+json
```

REST: założenia a praktyka

- Pełne REST (HATEOAS) rzadko wdrażane w całości
- Liczy się **spójność i przewidywalność**, nie dogmatyzm
- Kompromisy: zagnieżdżanie, batch endpoints, akcje
- Akcje poza CRUD: `POST /orders/7/cancel`
- Priorytet: czytelność dla zespołu i klientów API
- Trzymaj jeden styl w całym API
- Dokumentuj wyjątki od reguł (OpenAPI)

Ideał REST	Praktyka
HATEOAS	często pomijane
Czyste zasoby	endpointy akcji
1 żądanie = 1 zasób	batch dla wydajności
Wersja w nagłówku	wersja w URI (<code>/v1</code>)

Zasada: lepsze spójne i proste niż „czyste”, lecz niezrozumiałe.

HATEOAS i model dojrzałości Richardsona

- Poziom 0 – jeden URL, jedna metoda (RPC po HTTP)
- Poziom 1 – zasoby (osobne URI), wciąż jedna metoda
- Poziom 2 – metody HTTP + statusy (większość API)
- Poziom 3 – HATEOAS: odpowiedzi zawierają linki do akcji
- Linki prowadzą klienta przez stany aplikacji

```
{  
  "id": 7, "status": "new",  
  "_links": {  
    "self": { "href": "/orders/7" },  
    "cancel": { "href": "/orders/7/cancel" }  
  }  
}
```

Przykład: zasób `orders`

- Spójny zestaw endpointów dla pełnego CRUD
- Paginacja, filtrowanie i akcja poza CRUD

Metoda	URI	Akcja
GET	<code>/orders?status=new&page=1</code>	lista
GET	<code>/orders/7</code>	szczegóły
POST	<code>/orders</code>	utworzenie
PATCH	<code>/orders/7</code>	edycja
DELETE	<code>/orders/7</code>	usunięcie
POST	<code>/orders/7/cancel</code>	anulowanie

POST `/orders`

```
{ "userId": 42, "items": [{ "productId": 100, "qty": 2 }]}
```



Insomnia

Wprowadzenie

Czym jest Insomnia

Klient API do projektowania, debugowania i testowania interfejsów.

- Obsługa REST, GraphQL, gRPC, WebSocket i event-stream w jednym narzędziu
- Open source, lekki interfejs, szybki start bez zakładania konta
- **Spec-first**: Documents łączą OpenAPI, requesty i testy w jednym miejscu
- Środowiska, zmienne i **chaining** odpowiedzi między zadaniami
- Wbudowane testy, mocki oraz CLI `inso` do uruchamiania w CI/CD

Insomnia vs Postman

Cecha	Insomnia	Postman
Dane lokalne	Local Vault offline	wymuszona chmura
Wersjonowanie	Git Sync we własnym repo	chmura
Licencja	open source	własnościowa
Start	bez konta	konto wymagane

Prostszy i lżejszy, z pełną prywatnością danych dzięki Local Vault.

Instalacja i przegląd interfejsu

- Pobranie z insomnia.rest – Windows, macOS, Linux (instalator lub Homebrew)
- **Dashboard:** projekty, dokumenty i kolekcje w jednym widoku
- Wybór magazynu danych: **Local Vault**, **Cloud Sync** lub **Git Sync**
- Pasek żądania: metoda, URL, przycisk **Send** i panel odpowiedzi
- Zakładki requestu: **Params** , **Headers** , **Auth** , **Body** , **Scripts** , **Tests** , **Docs**
- Panel odpowiedzi: status, czas, rozmiar, podgląd, cookies i timeline
- **Command Palette** (**Ctrl/Cmd+K**) do szybkiej nawigacji

Przydatne skróty

Skrót	Akcja
Ctrl/Cmd+Enter	wyślij request
Ctrl/Cmd+K	command palette
Ctrl+Space	autouzupełnianie zmiennych
Ctrl/Cmd+N	nowy request

Tworzenie requestów

- Nowy request: wybór metody GET / POST / PUT / DELETE i adresu URL
- Query params, path variables i nagłówki w osobnych zakładkach
- Body: JSON, form-data, x-www-form-urlencoded, pliki, GraphQL
- Autoryzacja: Basic, Bearer, OAuth 2.0, AWS IAM, API Key, dziedziczona z folderu
- Zmienne {{ baseUrl }} zamiast wartości na sztywno (Ctrl+Space)
- Skrypty pre-request i after-response + asercje insomnia.test()
- Extra: import cURL, generowanie kodu klienta, wysyłka z opóźnieniem
- Send zwraca status, czas, rozmiar i podgląd odpowiedzi

Przykład

```
POST {{ baseUrl }}/api/users
Authorization: Bearer {{ token }}
Content-Type: application/json

{ "name": "Ada", "role": "admin" }
```

Organizowanie workspace i folderów

- Projekt grupuje powiązane dokumenty i kolekcje zespołu
- Kolekcja zbiera requesty jednego API w jednym miejscu
- Foldery: zagnieżdżanie requestów, wspólna autoryzacja, nagłówki i zmienne
- Chaining – przekazywanie wartości z odpowiedzi do kolejnych żądań
- Collection Runner uruchamia wiele requestów w zadanej kolejności
- Iteracje i opóźnienia w runnerze do powtarzalnych scenariuszy
- Czytelne nazwy i struktura ułatwiają pracę zespołu i CI/CD

Przykładowa struktura

```
Projekt: Sklep API
└─ Kolekcja: Users
    └─ Auth (wspólny Bearer token)
        ├── POST /login
        └── POST /refresh
    └─ Profile
        ├── GET /me
        └── PUT /me
```

Documents vs Collections

Document (spec-first)

- OpenAPI + requesty + testy w jednym pliku
- Auto-generowanie kolekcji wprost ze specyfikacji
- Live preview i linting z własnymi regułami
- Idealne do projektowania i dokumentacji API

Collection

- Sam zbiór requestów do wywołań i testów
- Foldery, autoryzacja per folder, chaining
- Collection Runner i import (Postman, cURL, HAR)
- Idealne do szybkich testów istniejącego API

Kiedy co wybrać

Kryterium	Document	Collection
Punkt startu	spec OpenAPI	gotowe API
Linting	tak	nie
Testy	tak	tak
Szybki start	wolniej	błyskawicznie

Document = projekt + dokumentacja, Collection = szybkie testy API.

Importowanie i eksportowanie kolekcji

- Import: Postman, OpenAPI/Swagger, cURL, HAR, WSDL
- Eksport: pojedynczy plik Insomnia v5 (JSON) lub OpenAPI
- Współdzielenie przez plik, Cloud Sync lub Git Sync
- Sharable link do dokumentacji API i mocków
- Migracja kolekcji między projektami i zespołami
- Wersjonowanie eksportu (JSON) w repozytorium Git
- Import: `Choose files > Scan > Import` w dashboardzie

Wspierane formaty importu

Źródło	Co przenosi
Postman	kolekcje, env, skrypty v2.0/2.1
OpenAPI/Swagger	spec + requesty
cURL	pojedyncze polecenia
HAR	sesje z przeglądarki
WSDL	usługi SOAP

Przechowywanie: Local, Cloud, Git

- Local Vault – dane tylko na urządzeniu, pełna prywatność i tryb offline
- Cloud Sync – szyfrowanie E2E (AES-GCM-256), sync między urządzeniami, zespół
- Git Sync – wersjonowanie projektu we własnym repozytorium Git
- Cloud: commit, push, revert, branch i dostęp zdalny z dowolnego miejsca
- Local + Git: idealne dla restrykcyjnych wymagań bezpieczeństwa
- Klucze generowane lokalnie; nazwa i id zasobu jawnie, payload szyfrowany

Porównanie

Magazyn	Prywatność	Współpraca	Offline
Local Vault	najwyższa	brak	tak
Cloud Sync	E2E	zespół	nie
Git Sync	własne repo	przez Git	tak

Wybór zależy od prywatności danych i modelu współpracy.

Klienci MCP w Insomnia

- Połączenie z serwerami Model Context Protocol (endpoint HTTP JSON-RPC)
- Odkrywanie narzędzi, promptów i zasobów AI gotowych do użycia
- Tworzenie: nowy klient → URL serwera → **Connect**
- Klient definiowany per projekt: Local, Git lub Cloud
- Autoryzacja: MCP Auth Flow (OAuth 2.0), Personal Access Tokens, aplikacje OAuth
- Mocki w chmurze do symulacji odpowiedzi podczas developmentu
- Walidacja: panele `Params`, `Headers`, `Events`, `Console` w czasie rzeczywistym
- Wsparcie **elicitation** i **sampling** w panelu odpowiedzi

Szybki start

Krok	Działanie
1	Utwórz klienta MCP
2	Wpisz URL serwera
3	Connect i autoryzacja
4	Wywołuj narzędzia i sprawdzaj Console

Praca z Insomniam – podstawy

Parametry, autoryzacja i projekt warsztatowy

Parametry żądania

- Query parameters – filtrowanie, sortowanie, paginacja
- Path variables – identyfikatory zasobów w URL
(/users/{id})
- Headers – Content-Type , Accept , Authorization
- Body (JSON) – payload dla POST / PUT / PATCH
- Tabela parametrów: szybkie włączanie/wyłączanie checkboxem
- Zmienne i template tags działają w URL, nagłówkach i body
- Foldery dzielą wspólne nagłówki między requestami
- Multipart/form-data dla uploadu plików

Element	Przykład
Query	?page=2&limit=20
Path	/users/{id}
Header	Accept: application/json
Body	{ "name": "Ada" }
<pre>POST /users?dryRun=true Content-Type: application/json { "email": "{{ email }}" }</pre>	

Autoryzacja

- Konfiguracja w zakładce **Auth** – formularz dopasowany do typu
- **Basic Auth** – login i hasło kodowane Base64 (Digest, Netrc)
- **Bearer Token** – token w nagłówku `Authorization`
- **OAuth 2.0** – pełne przepływy (Auth Code, Client Credentials)
- Auto-odświeżanie tokenu i podgląd ważności
- Inne: OAuth 1.0, AWS IAM v4, NTLM, Hawk, ASAP
- Foldery dziedziczą auth z rodzica – ustaw raz dla całej kolekcji
- Sekrety trzymaj w zmiennych, nie w surowym żądaniu

Typ	Zastosowanie
Basic	proste API, panele
Bearer	tokeny JWT/API
OAuth 2.0	logowanie SSO
API Key	nagłówek/query
<code>Authorization: Bearer {{ token }}</code>	
<code>Basic → base64(user:pass)</code>	

Zarządzanie środowiskami

- Środowiska to zestawy klucz–wartość (JSON) wstawiane do requestów
- Poziomy: globalne, kolekcji i folderu
- Base environment + sub-środowiska: `dev` , `staging` , `prod`
- Odwołania przez `Control+Space` lub `{{ baseUrl }}`
- Sub-środowiska dziedziczą bazę i nadpisują tylko różnice
- Szybkie przełączanie przez selektor w lewym górnym rogu
- Sekrety szyfrowane, odblokowywane kluczem vault
- Eksport/import bez sekretów dla bezpiecznego współdzielenia

```
// Base
{ "baseUrl": "https://api.dev", "ver": "v1" }

// prod nadpisuje
{ "baseUrl": "https://api.com" }
```

Poziom	Zakres
Globalne	wszystko
Kolekcja	requesty
Folder	grupa

Template tags i zmienne dynamiczne

- Wstaw przez `Control+Space` , kliknij tag aby skonfigurować
- **Faker** – losowe dane: imię, e-mail, UUID, timestamp
- **Response** – wartość z odpowiedzi innego żądania (chaining)
- **Request** – dane z bieżącego żądania
- Kodowanie: Base64, Hash, JWT, URL-encode
- Źródła: Prompt, Cookie, JSONPath, File, OS, Vault
- Własne tagi przez pluginy Insomnia
- Tryb surowej składni do ręcznej edycji wyrażeń

Tag	Wynik
<code>uuid v4</code>	losowy UUID
<code>now</code>	timestamp
<code>faker email</code>	dane testowe
<code>response</code>	z innego req
<pre>{% faker 'email' %} {% response 'body', 'req_1', '\$.token' %}</pre>	

Projektowanie endpointów

- Zasób bazowy: `users` z czytelnym, RESTful URI
- Metody HTTP zgodnie z semantyką operacji
- Filtrowanie, sortowanie i paginacja przez query params
- Spójne statusy `2xx` / `4xx` / `5xx` i format błędów
- Wersjonowanie API (np. `/v1`) od początku
- Liczba mnoga, bez czasowników w ścieżce
- Idempotentność: `PUT` zastępuje, `PATCH` aktualizuje
- Relacje przez zagnieżdżenie: `/users/{id}/orders`

Metoda	Ścieżka	Akcja
GET	<code>/users</code>	lista
POST	<code>/users</code>	utwórz
GET	<code>/users/</code>	szczegół
PUT	<code>/users/</code>	zastąp
DELETE	<code>/users/</code>	usuń
GET <code>/users?role=admin&page=2&limit=20</code>		

Tworzenie kolekcji Insomnia

- Kolekcja = zestaw requestów dla API użytkowników
- Foldery grupują logicznie (CRUD, autoryzacja)
- Foldery dzielą auth, nagłówki i zmienne środowiskowe
- Tworzenie od zera, import cURL lub OpenAPI/Postman
- Zmienne środowiskowe dla `baseUrl` i tokenu
- Eksport/import dla współpracy i wersjonowania
- Sync: Cloud, Local Vault lub Git
- Pin i opisy dla szybkiej nawigacji

```
Users API
├── Auth
│   └── POST /login
├── Users
│   ├── GET /users
│   ├── POST /users
│   └── DELETE /users/{id}
└── env: { baseUrl, token }
```

Import: New → cURL / OpenAPI

Wysyłanie żądań i analiza

- Wysyłanie GET / POST / PUT / DELETE i podgląd odpowiedzi
- Analiza statusu, nagłówków i body (JSON/raw)
- Sprawdzanie czasu odpowiedzi i rozmiaru
- Reuse wartości przez Response template tags
- Generowanie kodu klienta (cURL, JS, Python)
- Send after delay, repeat, timeline
- Filtrowanie JSON przez JSONPath/XPath
- Debugowanie nagłówków i payloadu w razie błędu

Sygnat

Co sprawdzić

Status

200/201/4xx

Time

< 300 ms

Size

rozmiar body

Headers

typ, auth

```
$.data[0].email ← JSONPath  
status: 201 · 142 ms · 1.2 KB
```

Symulacja błędów i poprawność

- Testy negatywne: brak tokenu, błędne body, zły id
- Walidacja statusów 4xx (klient) i 5xx (serwer)
- Przypadki graniczne i nieprawidłowe dane wejściowe
- Sprawdzanie komunikatów i formatu błędów
- Timeout żądania (domyślnie 30000 ms)
- Asercje na statusy, nagłówki i dane odpowiedzi
- Powtarzalność: każdy test od czystego stanu
- Negatywy obok pozytywów w Collection Runner

Scenariusz	Status
Brak tokenu	401
Złe dane	400/422
Zły id	404
Błąd serwera	500

```
insomnia.test('Brak tokenu → 401', () =>
  insomnia.expect(insomnia.response.code)
    .to.eql(401))
```

Testowanie API w Insomnia

Asercje, chaining, skrypty i runner

Asercje: statusy, nagłówki, dane

- Asercje piszemy w After-response script w stylu Chai
- Status: `insomnia.response.code` → kod odpowiedzi
- Nagłówki: sprawdzaj `Content-Type` i obecność kluczy
- Dane: waliduj strukturę i wartości w body (JSON)
- Grupuj sprawdzenia w `insomnia.test()` z czytelną nazwą
- Sprawdzaj czas: `insomnia.response.responseTime`
- Wyniki testów widoczne w zakładce Tests

```
insomnia.test('Status 200 + JSON', () => {  
  insomnia.expect(insomnia.response.code)  
    .to.eql(200)  
  insomnia.response.to.have  
    .header('content-type')  
  const b = insomnia.response.json()  
  insomnia.expect(b.id).to.be.a('number')  
})
```

Testy sekwencyjne (chaining)

- Łącuchowanie używa odpowiedzi w kolejnym żądaniu
- Wyślij pierwszy request, by wygenerować odpowiedź
- W kolejnym wpisz `response` → tag **Response**
- Wybierz **Body Attribute** i wyciągnij przez **JSONPath**
- Przykład filtra: `$.[0].id` lub `$.token`
- Trigger steruje odświeżaniem wartości między żądaniami
- Scenariusz: login → pobierz ID → aktualizuj

```
POST /login → token
GET  /users → id
PUT  /users/{{ id }}
      Authorization: Bearer {{ token }}
```

Krok	Wynik
login	token
list	id
update	200

Skrypty pre/post (Hooks)

- Pre-request: ustaw zmienne, edytuj nagłówki/body/auth
- After-response: testy z `insomnia.test()` / `expect()`
- Dostęp do zmiennych: base, collection, folder
- Wbudowane: chai, lodash, moment, uuid, crypto-js
- Dodatkowe żądania: `insomnia.sendRequest()`
- Import skryptów z Postman v2.0/v2.1
- Logowanie do Console przez `console.log()`

```
// Pre-request
insomnia.environment.set('ts', Date.now())

// After-response
insomnia.test('OK', () =>
  insomnia.expect(insomnia.response.code)
    .to.eql(200))
```


Dane dynamiczne i zmienne

- Zmienne środowiskowe oddzielają dane od żądań (`{{ base_url }}`)
- Środowiska bazowe i podrzędne: dev / staging / prod
- Dane dynamiczne generuj skryptem: `uuid` , `Date.now()` , `faker`
- Zapis i odczyt: `insomnia.environment.set/get`
- Template tags wstrzykują wartości do URL, nagłówków i body
- Wartości z odpowiedzi zasilają kolejne testy (chaining)
- Unikalne e-maile/ID zapobiegają konfliktom danych
- Sekrety w vault, nie w kodzie żądania

```
insomnia.environment.set('email',  
  `user_${Date.now()}@test.dev`)  
// body: { "email": "{{ email }}" }
```

Funkcja	Wynik
<code>uuid</code>	losowy ID
<code>Date.now()</code>	znacznik
<code>faker</code>	dane testowe

Collection Runner

- Uruchom wiele żądań w ustalonej kolejności i porównaj wyniki
- Otwórz kolekcję → kliknij nazwę → **Run Collection**
- Ustaw kolejność (drag & drop) i wybierz żądania
- Iteracje i opóźnienie powtarzają sekwencję
- Plik danych (CSV/JSON) zasila kolejne iteracje
- Podgląd żądań i odpowiedzi w zakładce **Console**
- Raport: liczba przejść, błędów i czas wykonania
- Automatyzacja w CI/CD: `inso run collection`

Opcja	Efekt
Iterations	powtórzenia
Delay	przerwa ms
Data file	dane wejść.
Tests	asercje

```
inso run collection \  
  --env prod --reporter junit
```

Dokumentacja REST API

OpenAPI, generowanie i udostępnianie

OpenAPI (Swagger)

– podstawy i integracja

- Standardowy, czytelny dla maszyn format opisu API w YAML/JSON
- Insomnia Document = spec + requesty + testy w jednym workspace
- Podejście **spec-first**: najpierw projekt, potem implementacja
- Import lub tworzenie specyfikacji w widoku **Design**
- Żywy podgląd specyfikacji w prawym panelu (**Preview**)
- Auto-kategoryzacja: `servers` , `request` `body` , `schemas`
- **Generate Collection** tworzy requesty wprost ze specyfikacji

```
---
openapi: '3.0.3'
info:
  title: 'User API'
  version: '1.0.0'
servers:
  - url: 'https://api.example.com'
paths:
  /users:
    get:
      summary: 'Lista użytkowników'
      responses:
        '200':
          description: 'OK'
```

Jeden plik = kontrakt dla front-endu, back-endu, QA i dokumentacji.

Generowanie dokumentacji

- Dokumentacja powstaje automatycznie ze specyfikacji OpenAPI
- Interaktywny renderer (panel **Preview**) pokazuje endpointy i schematy
- Edytor i podgląd aktualizują się na żywo podczas pisania specyfikacji
- **Generate Collection** tworzy gotowe requesty wprost z dokumentu
- Snippety kodu w 12+ językach (cURL, JS, Python, Go, PHP...)
- Pola `summary`, `description`, `examples` zasilają opisy w podglądzie
- Spójność: jedno źródło prawdy dla front-endu, back-endu i QA

Typowy przepływ pracy:

1. Napisz/zaimportuj spec OpenAPI w widoku **Design**
2. Sprawdź endpointy w panelu **Preview**
3. **Generate Collection** → testy i requesty w **Debug**
4. Eksport / Sharable Link dla zespołu

```
inso export spec "User API" --output openapi.yaml
```

Aktualizacja specyfikacji = aktualizacja dokumentacji bez ręcznej pracy.

Komentarze, opisy i struktura projektu

- Opisy endpointów w polach `summary` (krótko) i `description` (Markdown)
- Dokumentacja parametrów: `path` , `query` , `header` , `request body`
- Przykłady żądań i odpowiedzi w `examples` — szybciej zrozumiesz API
- Schematy w `components/schemas` = modele wielokrotnego użytku (`$ref`)
- Czytelna struktura: `servers` , `paths` , `components` , `tags`
- Wersjonowanie i współpraca przez Git Sync (review, historia, PR)

```
paths:
  /users/{id}:
    get:
      summary: Pobierz użytkownika
      description: Zwraca dane konta po **ID**
      parameters:
        - { name: id, in: path, required: true, schema: {
      responses:
        '200': { description: OK }
```

`tags` grupują endpointy, a `$ref` eliminuje duplikację schematów.

Linting specyfikacji (Spectral)

- Automatyczna walidacja specyfikacji OpenAPI/AsyncAPI
- Błędy wskazane wg typu, numeru linii i opisu — łatwa nawigacja
- Wbudowane reguły (`spectral:oas`) + własne zasady w `.spectral.yaml`
- Konsola błędów w Insomnia ułatwia szybkie poprawki przed commitem
- Spójność i jakość API jeszcze przed wdrożeniem (shift-left)
- Linting w pipeline dzięki `inso lint spec` / Spectral CLI

```
# .spectral.yaml
extends: spectral:oas
rules:
  operation-tags: error
  info-contact: warn
```

```
inso lint spec "User API" --ci # exit ≠ 0 zatrzyma pipeline
```



Reguły jako error blokują build, jako warn tylko ostrzegają.

Udostępnianie API

- Eksport dokumentacji jako plik OpenAPI/Swagger (YAML/JSON)
- Sharable link do specyfikacji dla zespołu i interesariuszy
- Synchronizacja przez Cloud Sync lub Git Sync (repo z zespołem)
- Import: Postman, OpenAPI, cURL, HAR, WSDL, Insomnia v4
- Plik OpenAPI jako kontrakt dla front-endu, back-endu i QA
- Łatwa integracja z CI/CD i narzędziami zewnętrznymi (inso)

Sposoby dzielenia się projektem:

Metoda	Zastosowanie
Export (.yaml)	Backup, kontrakt, import gdzie indziej
Sharable link	Szybki podgląd dla osób bez konta
Cloud Sync	Współpraca w czasie rzeczywistym
Git Sync	Review, historia, branże, CI/CD

```
inso export spec "User API" --output openapi.yaml
```


Automatyzacja i CI/CD

Inso CLI, pipeline'y i mockowanie

Debugowanie i logi w Insomnia

- Konsola (timeline) pokazuje pełny request/response: nagłówki, body, czasy
- Analiza statusów `4xx` / `5xx` i komunikatów błędów wprost z odpowiedzi
- Tryb `verbose` dla `inso` (`--verbose`) ujawnia szczegóły wywołań w CI
- Podgląd zmiennych środowiskowych i renderowanych wartości przed wysłaniem
- Logowanie czasów odpowiedzi pomaga wykryć wąskie gardła i timeouty
- Weryfikacja TLS/proxy gdy żądania nie docierają do serwera
- Tab **Console** dla skryptów: `console.log()` z pre/post-request scripts

```
inso run test "User API" --env Prod --verbose --ci
```

Objaw	Najczęstsza przyczyna
401/403	Zły token / wygaśnięcie środowisko
ECONNREFUSED	Serwer nie działa lub zły URL
timeout	Sieć/proxy, <code>requestTimeout</code>

Insomnia CLI (inso)

- Insomnia w terminalu i w CI/CD — pełna automatyzacja bez GUI
- Instalacja: `brew install inso`, `zip/tar` lub `docker pull kong/inso:latest`
- Uruchamianie testów i kolekcji bez interfejsu graficznego
- Lint i eksport specyfikacji OpenAPI z linii komend
- Źródło danych: katalog `.insomnia` (Git sync) lub `--workingDir / --src`
- Konfiguracja w pliku `.insorc` (flagi, `requestTimeout`, skrypty)
- Identyfikator zasobu po nazwie lub ID; `--ci` wyłącza prompty

```
inso run test "User API" --env "Prod" --ci
inso lint spec "User API"
inso export spec "User API" --output openapi.yaml
```

Komenda	Zastosowanie
<code>run test</code>	Asercje / regresja
<code>lint spec</code>	Walidacja OpenAPI
<code>export spec</code>	Eksport kontraktu

Integracja z CI/CD

- `inso` działa w każdym pipeline (Linux, macOS, Windows, Docker)
- Flaga `--ci` wyłącza interaktywne prompty i zwraca kody wyjścia
- Pass/fail steruje przejściem etapu — czerwony build przy błędzie
- **GitHub**: gotowa akcja `kong/setup-inso@v2`
- **GitLab/Jenkins**: ten sam obraz `kong/inso` w stage testowym
- Lint + testy jako bramka jakości przed merge/deploy

```
- uses: kong/setup-inso@v2
  with: { inso-version: 11 }
- run: inso lint spec "User API" --ci
- run: inso run test "User API" --env UnitTest --ci
```

GitLab CI

```
test-api:
  image: kong/inso:latest
  script:
    - inso lint spec "User API" --ci
    - inso run test "User API" --ci
```

Jenkins

```
sh 'docker run kong/inso run test "User API" --ci'
```

Testy regresji i smoke tests

- **Smoke test** — szybka weryfikacja kluczowych endpointów po deployu
- **Regresja** — pełny zestaw asercji chroniący przed cofnięciem funkcji
- Uruchamiane przez `inso run test "doc" --env "env"` w CI/CD
- `--bail` przerywa po pierwszym błędzie; `--testNamePattern` filtruje testy
- Osobne środowiska (smoke vs full) i odrębne harmonogramy uruchamiania
- Kod wyjścia $\neq 0$ blokuje pipeline i sygnalizuje regresję
- Smoke po deployu, regresja nocą (cron) na pełnym środowisku

Cecha	Smoke	Regresja
Zakres	Krytyczne	Wszystko
Czas	Sekundy	Minuty
Kiedy	Po deployu	Nightly / PR

```
inso run test "User API" --env Smoke --bail --ci
inso run test "User API" --env Full --ci
```

Mockowanie API (Insomnia Designer)

- Mock serwer zastępuje realne endpointy — praca równolegle z backendem
- **Create > Mock Server**, nazwa, wybór cloud/self-hosted, **Create**
- Definiujesz **route**: ścieżkę, metodę, body, nagłówki i status
- Auto-generacja mocków z URL, OpenAPI lub tekstu (prompt)
- **Show URL** daje endpoint, **Test** weryfikuje odpowiedź
- Wymaga konta Insomnia — niedostępne w lokalnym Scratch Pad
- Idealne dla front-endu, gdy backend jeszcze nie istnieje

Przykładowa route:

Pole	Wartość
Metoda	GET
Ścieżka	/users/1
Status	200
Body	{ "id": 1, "name": "Ada" }

Frontend rozwija się bez czekania na gotowy backend.

Mock: Cloud vs Self-hosted

Cloud-hosted

- Błyskawiczna konfiguracja, brak infrastruktury
- Hosting po stronie Insomnia (Kong)
- Limity zapytań zależne od planu
- Jedna odpowiedź na route
- Najlepsze do prototypów i demo

Create > Mock Server > Cloud

Self-hosted

- Własny serwer Mockbin, pełna kontrola
- Obraz `kong/insomnia-mockbin-self-hosted` + Redis
- Wdrożenie: Node.js (`:8080`), Docker, K8s
- Dane wewnątrz organizacji (compliance)
- Wyłącz proxy: **Preferences > Proxy**

```
docker run -p 8080:8080 \
  kong/insomnia-mockbin-self-hosted
```

Dynamiczne mockowanie

- Mock Body renderowane jako Liquid w momencie żądania
- `req` daje dostęp do nagłówków, query, ścieżki i body requestu
- `faker.<zmienna>` generuje losowe imiona, e-maile, UUID, daty
- Bezpieczne tagi: `assign`, `if`, `unless`, `raw`; filtr `default`
- Wartości domyślne dla realistycznych, kontekstowych odpowiedzi
- Opcje: wyślij, generuj kod, opóźnienie, powtarzanie w interwale
- Odpowiedź zależna od wejścia — testy edge case bez backendu

```
{  
  "name": "{{ req.body.name | default: \"George\" }}",  
  "id": "{{ faker.uuid }}",  
  "email": "{{ faker.email }}",  
  "createdAt": "{{ faker.iso8601 }}"  
}
```

Liquid + faker = mock, który zachowuje się jak prawdziwe API.

Bezpieczeństwo REST API

Autoryzacja, ochrona i dobre praktyki

Autoryzacja i uwierzytelnianie

- Uwierzytelnianie = kim jesteś; autoryzacja = do czego masz dostęp
- Basic Auth – login i hasło, tylko po HTTPS, do prostych przypadków
- Bearer Token / API Key – token w nagłówku, łatwe odwołanie i rotacja
- OAuth2 – delegacja dostępu, access + refresh token, usługi zewnętrzne
- JWT – samozawierający token z claims, weryfikacja podpisu bez sesji

```
Authorization: Bearer {{ token }}
```

Metoda	Zastosowanie
Basic	Wewnętrzne, szybkie testy
API Key	Integracje serwer-serwer
OAuth2	Logowanie Google/GitHub
JWT	Bezstanowe sesje, mikroserwisy

W Insomnia:

- Zakładka *Auth* + zmienne na tokeny
- Sekretów nie commituj — **local vault** / private environment
- **Git Sync**: konfig współdzielony, sekrety lokalnie

Ochrona API

- **Rate limiting** – limit żądań na klucz/IP w oknie czasu, status 429
- **Throttling** – spowalnianie zamiast blokady przy nadmiarze ruchu
- **Backoff** po stronie klienta – ponawianie z rosnącym opóźnieniem
- Ochrona przed DoS, brute force i wyciekami danych
- Limity per użytkownik, plan i endpoint
- Testuj scenariusze 429 w Insomnia + nagłówek `Retry-After`

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
```

Nagłówek

Znaczenie

`X-RateLimit-Limit`

maks. żądań

`X-RateLimit-Remaining`

pozostało

`X-RateLimit-Reset`

reset okna

`Retry-After`

ponów za N s

W Insomnia:

- Zmienne na limity per środowisko
- `inso run test` weryfikuje 429
- Git Sync: testy w pull request

Dobre praktyki

- Walidacja wejścia – schemat, typy, sanityzacja, status 400
- CORS – jawna lista origin, metod i nagłówków; unikaj * w produkcji
- Ograniczanie metod – tylko potrzebne, 405 dla reszty
- Zawsze HTTPS; nie zwracaj stack trace ani szczegółów błędów
- Zasada least privilege dla tokenów i ról
- Sekrety w local vault / private environment, nie w repo

```
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Methods: GET, POST
```

Element	Dobrze	Źle
Origin	lista domen	*
Błąd	kod + opis	stack trace
Token	wąski zakres	pełny dostęp
HTTP	wymuszony	mieszany

W Insomnia:

- Walidacja inso lint spec w CI/CD
- Sekrety per środowisko, audyt w Git Sync

Utrzymanie i rozwój REST API

Wersjonowanie, kompatybilność i dobre praktyki

Wersjonowanie i kompatybilność

- Wersja w URL – `/api/v1/ ...` – proste i czytelne
- Wersja w nagłówku – `Accept: application/vnd.api+json; v=2`
- Zgodność wsteczna – nowe pola opcjonalne, nie usuwaj istniejących
- Zmiany niełamające vs łamiące (breaking) → podbij wersję główną
- SemVer: `MAJOR.MINOR.PATCH` dla kontraktu API
- Kolekcje per wersja; Git Sync śledzi historię specyfikacji

```
GET /api/v1/users
Accept: application/vnd.api+json; v=2
```

Zmiana	Przykład	Wersja
Nowe pole opcjonalne	+ phone	MINOR
Poprawka błędu	fix 500	PATCH
Usunięcie pola	- email	MAJOR
Zmiana typu	id int→uuid	MAJOR

-  dodawaj,  nie usuwaj bez deprecjacji
- Tag w Git = zamrożony kontrakt

Deprecacja i migracje

- Oznaczaj przestarzałe pola/endpointy nagłówkiem `Deprecation` i `Sunset`
- Określ okres przejściowy i datę wyłączenia starej wersji
- Plan migracji – przewodnik, mapowanie pól, równoległe wersje
- Komunikacja: changelog, mail, dokumentacja, dev portal
- Monitoruj użycie starych wersji przed wygaszeniem
- Wersjonuj zmiany w Git Sync — pełna historia kontraktu

```
HTTP/1.1 200 OK
Deprecation: true
Sunset: Wed, 31 Dec 2026 23:59:59 GMT
Link: </api/v2/users>; rel="successor-version"
```

- ● v1 i v2 działają równoległe przez okres przejściowy
- ● ostrzeżenie w odpowiedzi + termin sunset
- ● dopiero po terminie: wyłączenie i `410 Gone`

Testy zgodności

- Walidacja specyfikacji OpenAPI (`inso lint spec`) w pipeline
- Testy kontraktowe – odpowiedzi zgodne ze schematem
- Testy regresji i smoke testy po każdej zmianie
- Automatyzacja w CI/CD przez `inso run test`
- Wykrywanie zmian łamiących **przed** wdrożeniem
- Git Sync: review specyfikacji w pull request

```
# w CI/CD
inso lint spec
inso run test "Smoke v1" --env prod
```

-  zielony pipeline = bezpieczne wdrożenie
-  brak testów = breaking change wykryty u klienta
-  spec jako kontrakt → testy pilnują zgodności

Organizacja projektu

- Konwencje nazw – zasoby w l. mnogiej, kebab-case , spójność
- Folderowanie kolekcji wg domen i wersji
- Środowiska dev/staging/prod jako zmienne środowiskowe
- Sekrety w local vault / private environment, nie w plikach
- Git Sync – własne repo (GitHub/GitLab) zamiast cloud, review zmian
- Jeden styl błędów, paginacji i nagłówków w całym API

```
collection/  
  users/  v1/  v2/  
  orders/ v1/  
env: dev | staging | prod
```

Element	Konwencja
Zasoby	/users , /orders
Ścieżki	kebab-case
Wersje	/v1 , /v2
Pola JSON	camelCase
Sekrety	local vault

- Spójność > kreatywność
- Onboarding szybszy dzięki Git Sync

Dobre i złe praktyki

-  Spójne nazwy, statusy HTTP i format błędów
-  Wersjonowanie, paginacja, dokumentacja OpenAPI
-  HTTPS, walidacja, rate limiting
-  Sekrety w local vault, audyt przez Git Sync
-  Testy `inso run test` w CI/CD
-  Czasowniki w URL (`/getUsers`), `200` dla błędu
-  Brak wersji, breaking changes bez ostrzeżenia
-  Tokeny i hasła commitowane do repo

Aspekt	 Dobrze	 Źle
URL	<code>/users</code>	<code>/getUsers</code>
Status	<code>404</code>	<code>200</code> +błąd
Wersja	<code>/v1</code>	brak
Sekret	vault	repo
Testy	CI/CD	ręczne

 Spójność i przewidywalność to fundament dobrego API.

Przypadki z życia

✓ Co działa

- Stabilny kontrakt + testy w CI/CD = bezpieczne wdrożenia
- Git Sync i review specyfikacji w pull request
- Mocki przyspieszają pracę frontendu przed backendem
- `inso` automatyzuje testy regresji przy każdym merge
- Wersja `/v2` + `sunset` → płynna migracja klientów

✗ Co zawodzi

- Brak limitów → przeciążenie i awarie produkcji
- Usunięcie pola bez deprecjacji → zerwane integracje
- Sekrety w repo → wyciek i pilna rotacja tokenów
- Brak monitoringu → błędy widzi klient, nie zespół

💡 Lekcja: dokumentacja, testy i monitoring to inwestycja, nie koszt
— zespół migrował 30 klientów z zerem awarii.

Czytelność i użyteczność API

- Intuicyjne, przewidywalne URI i spójne nazewnictwo
- Czytelne komunikaty błędów z kodem i opisem
- Dokumentacja OpenAPI z przykładami i opisami
- Sensowne wartości domyślne, paginacja, filtrowanie
- Sharable link i kolekcje ułatwiają onboarding
- Łatwa integracja: mniej zaskoczeń = mniej zgłoszeń

```
GET /users?page=2&limit=20&sort=name
```

-  jednolite filtry: `?page=` , `?limit=` , `?sort=`
-  przykłady gotowe do skopiowania w docs
-  Kolekcje w **Git Sync** = zespół startuje w minuty

Antywzorce i jak ich unikać

- ❌ Czasowniki w ścieżkach → ✅ zasoby + metody HTTP
- ❌ Status `200` z błędem → ✅ właściwe `4xx` / `5xx`
- ❌ Brak wersjonowania → ✅ `/v1` , deprecjacja, sunset
- ❌ Sekrety w repo → ✅ local vault / private environment
- ❌ Brak walidacji i limitów → ✅ walidacja + rate limiting
- ❌ Ręczne testy → ✅ automatyzacja `inso` w CI/CD

Antywzorzec	Rozwiązanie
<code>/createUser</code>	POST <code>/users</code>
<code>200</code> przy błędzie	<code>400</code> / <code>404</code>
zmiana bez wersji	<code>/v2</code> + sunset
hasło w repo	vault + env
brak limitów	<code>429</code> + backoff

💡 **Git Sync** = review w PR łapie antywzorce zanim trafią na produkcję.